

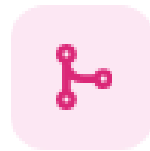
Sistema de Préstamos en Memoria

Arquitectura Hexagonal (Puertos y Adaptadores)

Yury Gabriela Alarcón Bustos
Stephany Yicell Galindo Cañas
Andres Felipe Paez Diaz
Cristian David Vacca Alvarado

HERRAMIENTA UTILIZADA Gemini (Asistente de programación)	LA USAMOS PARA Diseñar la estructura base de directorios, definir contratos e implementar la lógica aislada.	PARTES APOYADAS POR IA Generación de código Python (sin librerías), justificación y diagrama de 6 bloques.
VERIFICACIONES REALIZADAS Ejecutar pruebas para validar casos de aceptación y auditar código asegurando que el núcleo es agnóstico.		DECISIONES ARQUITECTÓNICAS 1) Módulo abstracto. 2) Inyección desde main. 3) Inventario en memoria.

El Problema Común



Alto Acoplamiento (Código Espagueti)

Normally, mezclamos la lógica de negocio con la persistencia de datos o la interfaz de usuario directamente en las mismas capas.



Consecuencia Directa

Si queremos cambiar de consola a una aplicación web, tenemos que reescribir y modificar el corazón del sistema, arriesgándonos a dañar las reglas de negocio.



Concepto clave: Dependencias rígidas y difíciles de escalar.

Diagrama Arquitectónico (6 Bloques)

Cumplimiento estricto de seis bloques y dependencias hacia adentro



Recorrido por el Código y Estructura

📁 Estructura del Proyecto

```
nucleo/  
├── modelo.py  
└── servicio_prestamos.py  
  
puertos/  
├── inventario_port.py  
└── notificador_port.py  
  
adaptadores/  
├── inventario_memoria.py  
└── consola.py  
  
main.py (Orquestador)
```

⚙️ Detalle Central del Núcleo

La clase **ServicioPrestamos** inyecta **InventarioPort** y **NotificadorPort** en su constructor. Las validaciones de aprobación y rechazo usan estos puertos, logrando ser **100% agnósticas** a cómo se guarda la información o cómo se muestra en pantalla.

```
class ServicioPrestamos:  
    def __init__(self, inv: InventarioPort, notif: NotificadorPort):  
        # Inyección de abstracciones puras  
        self.inv = inv  
        self.notif = notif
```

Pruebas de Aceptación

Caso de Uso	Acción Ejecutada	Resultado Obtenido	Estado
Caso 1	Inventario inicial (2 portátiles, 1 tableta)	Estado inicial visible en consola.	EXITOSO
Caso 2	Solicitud aprobada (Ana solicita 1 portátil)	"Aprobada; queda 1 portátil."	EXITOSO
Caso 3	Solicitud rechazada (Juan solicita 2 tabletas)	"Rechazada; queda 1 tableta."	EXITOSO
Caso 4	Reporte final (Ejecutar cierre del programa)	Inventario final y solicitudes procesadas impresas.	EXITOSO

Trade-offs (Decisiones)

✔ Ventajas

Agnosticismo Total: Cambiamos la consola por API REST sin tocar una sola regla de negocio.

Testing Simplificado: Aislar componentes e inyectar datos falsos (Mocks) es trivial.

Código Longevo: El núcleo no envejece al ritmo de los frameworks externos.

⚖ Desventajas

Curva de Aprendizaje: Requiere entender el Principio de Inversión de Control e interfaces.

Boilerplate Inicial: Para sistemas muy pequeños, crear la distribución de puertos y adaptadores suma tiempo extra y sobrecarga de archivos.

Conclusión: La inversión inicial en estructura garantiza flexibilidad y facilidad de mantenimiento.